ASSIGNMENT 3 REPORT

Multi-class Classification using a Fully Connected Neural Network (FCNN) on Fashion-MNIST

Optimizer Comparison, Convergence and Classification Performance

Course: Large Language Modeling (LLM)
Assignment 3 • Task 2

1. Objective

The objective of this task is to study how different backpropagation optimizers affect the training and classification performance of fully connected neural networks (FCNNs) on a five-class subset of the Fashion-MNIST dataset. Three FCNN architectures with 3, 4 and 5 hidden layers are trained using six optimization methods: SGD, Batch Gradient Descent, Momentum, Nesterov Accelerated Gradient (NAG), RMSProp and Adam.

The comparison focuses on two practical aspects: (1) the number of epochs required to satisfy the convergence criterion and (2) training and validation accuracy. The architecture is selected using validation accuracy, while the held-out test set is used only for the final evaluation.

2. Dataset and Preprocessing

Fashion-MNIST contains 28 × 28 grayscale images belonging to ten clothing categories. For this assignment, exactly five classes were retained: Trouser, Sandal, Sneaker, Bag and Ankle boot. A total of 35,000 selected samples were used. Each image was normalized by dividing pixel values by 255 and then flattened from 28 × 28 into a 784-dimensional input vector.

Item	Value
Dataset	Fashion-MNIST
Selected classes	Trouser, Sandal, Sneaker, Bag, Ankle boot
Selected samples	35,000
Image size	28 × 28 grayscale
Input size	784
Normalization	Pixel values / 255 → [0, 1]
Final split	64% train (22,400), 16% validation (5,600), 20% test (7,000)
Split method	Stratified; random seed = 42
Output classes	5
Training device	CUDA

The split preserves the relative representation of all five classes. The test set is kept separate from model selection, so validation accuracy is used to identify the best architecture–optimizer combination.

3. FCNN Architectures

All three networks use fully connected layers with ReLU activation in the hidden layers and a five-unit output layer corresponding to the five selected classes. The architectures differ in depth and hidden-layer width.

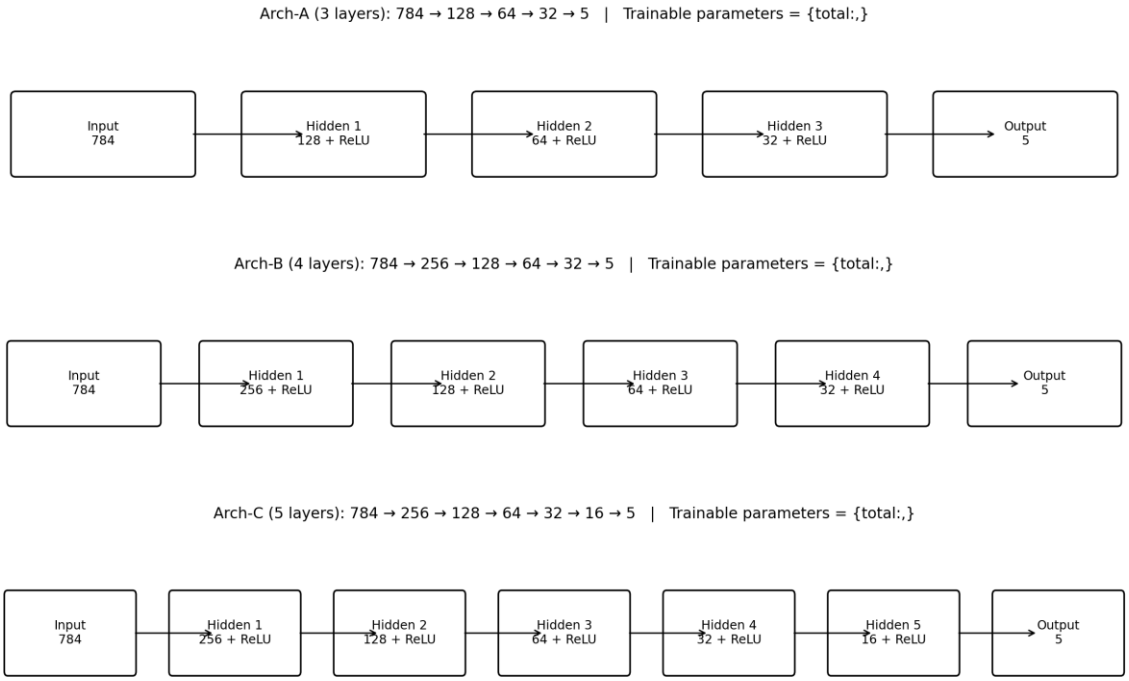


Figure 1. FCNN architectures used in the experiment.

Architecture	Layer structure	Trainable parameters
Arch-A	784 → 128 → 64 → 32 → 5	110,981
Arch-B	784 → 256 → 128 → 64 → 32 → 5	244,357
Arch-C	784 → 256 → 128 → 64 → 32 → 16 → 5	244,805

Trainable parameters include weights and biases for every fully connected layer, using Σ(n_in × n_out + n_out). Arch-A is the smallest model, while Arch-C is the deepest. Interestingly, Arch-C has only 448 more parameters than Arch-B because its final hidden layer reduces the dimensionality from 32 to 16 before the five-class output.

4. Loss Function and Optimization Setup

Cross-entropy loss is used because the task is multi-class classification. The final layer produces class scores which are converted into probabilities using softmax. ReLU is used between the fully connected hidden layers.

Key equations:

- Cross-entropy: $L = -(1/N)\ \Sigma_i \log p_{i,y_i}$
- Softmax: $p_k = \exp(z_k) / \Sigma_j \exp(z_j)$
- Fully connected layer: $z^l = W^l a^{l-1} + b^l$
- Hidden activation: $a^l = \mathrm{ReLU}(z^l)$

Parameter	Value used in completed output
Learning rate η	0.001
Momentum (Momentum / NAG)	0.9
RMSProp β (alpha)	0.99
RMSProp ε	1×10^{-8}
Adam β_1	0.9
Adam β_2	0.999
Adam ε	1×10^{-8}
Stopping threshold	1×10^{-4}
Maximum epochs	100 for non-Batch-GD; 5000 for Batch GD
Mini-batch size	16 for SGD, Momentum, NAG, RMSProp and Adam
Batch GD size	22,400 (all training samples)

The stopping criterion used in the completed output is $|E_t - E_{t-1}| < 1 \times 10^{-4}$, where E is the average training cross-entropy error. This stops training when successive epoch errors change by less than the specified threshold.

Implementation note: the assignment specification states batch_size = 1 for SGD, Momentum, NAG, RMSProp and Adam, whereas the supplied completed output records a mini-batch size of 16. The results in this report are reproduced from the supplied completed output. If strict submission compliance is required, the notebook should be checked and the batch size aligned with the assignment specification before final submission.

5. Convergence and Accuracy Results

The following table compares all 18 architecture–optimizer combinations. A smaller epoch count indicates faster satisfaction of the stopping criterion, while higher validation accuracy indicates better generalization on the validation set.

Architecture	Optimizer	Epochs	Train Acc. (%)	Validation Acc. (%)
Arch-A (3 layers)	SGD	69	97.05	95.89
Arch-A (3 layers)	Batch GD	2	20.0	20.0
Arch-A (3 layers)	Momentum	30	98.99	96.89
Arch-A (3 layers)	NAG	30	98.8	96.77
Arch-A (3 layers)	RMSProp	49	99.71	97.18
Arch-A (3 layers)	Adam	25	99.46	97.36
Arch-B (4 layers)	SGD	84	97.75	95.96
Arch-B (4 layers)	Batch GD	2	20.0	20.0
Arch-B (4 layers)	Momentum	45	99.44	97.11
Arch-B (4 layers)	NAG	37	98.42	96.43
Arch-B (4 layers)	RMSProp	33	99.47	97.21
Arch-B (4 layers)	Adam	22	98.87	96.98
Arch-C (5 layers)	SGD	73	97.03	94.77
Arch-C (5 layers)	Batch GD	2	20.0	20.0
Arch-C (5 layers)	Momentum	27	99.09	96.82
Arch-C (5 layers)	NAG	44	99.49	97.12
Arch-C (5 layers)	RMSProp	64	99.54	96.82
Arch-C (5 layers)	Adam	54	99.69	97.25

Across the non-Batch-GD methods, validation accuracy remains in the mid-to-high 90% range. The strongest validation result is 97.36% from Arch-A with Adam. Arch-B with RMSProp reaches 97.21%, while Arch-C with Adam reaches 97.25%.

Batch Gradient Descent reaches the stopping criterion in only two epochs for every architecture, but its recorded training and validation accuracy remain at 20.00%. With five output classes, 20% corresponds to chance-level accuracy, so the recorded Batch-GD result does not provide useful classification performance despite its very small epoch count.

6. Training Error Curves and Convergence Analysis

The error curves show how average training cross-entropy changes with epochs. Batch GD is shown separately from the zoomed views because its behavior dominates the scale of the full plots.

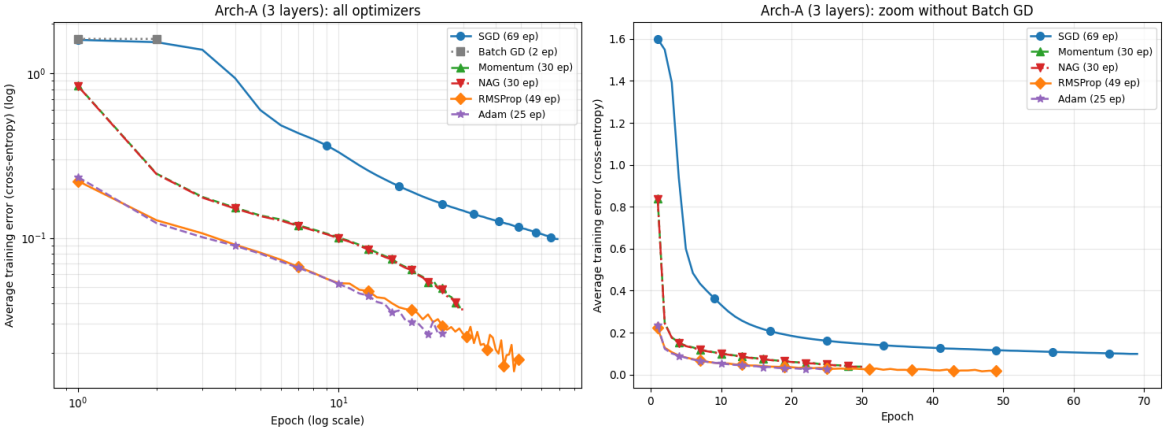


Figure 2. Arch-A training error curves for the six optimizers.

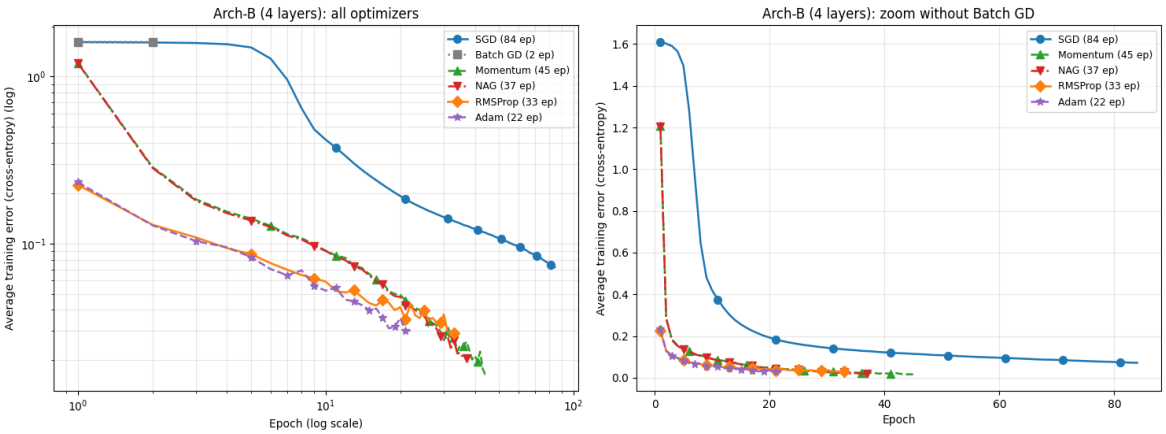


Figure 3. Arch-B training error curves for the six optimizers.

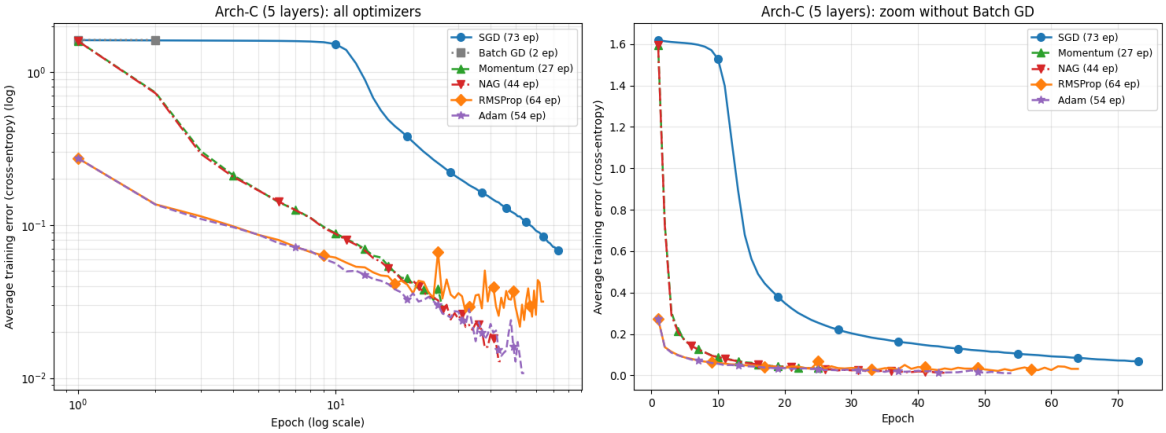


Figure 4. Arch-C training error curves for the six optimizers.

- Arch-A: Adam converges in 25 epochs and gives the highest validation accuracy (97.36%) among all runs.
- Arch-B: Adam converges fastest among the non-Batch-GD runs at 22 epochs, while RMSProp gives the highest validation accuracy for this architecture (97.21%).

- Arch-C: Momentum converges fastest among the non-Batch-GD runs at 27 epochs, whereas Adam gives the highest validation accuracy for this architecture (97.25%).
- SGD generally requires more epochs than Adam, Momentum and NAG, indicating slower convergence under the selected settings.
- Higher training accuracy does not automatically produce higher validation accuracy; therefore validation accuracy is the appropriate criterion for selecting the final model.

7. Selection of the Best Model

The best model is selected using the highest validation accuracy across all 18 architecture–optimizer combinations. The selected model is Arch-A with Adam.

Item	Result
Best architecture	Arch-A (3 hidden layers): 784 → 128 → 64 → 32 → 5
Best optimizer	Adam
Validation accuracy	97.36%
Training accuracy	99.46%
Test accuracy	97.64%
Trainable parameters	110,981
Selection criterion	Highest validation accuracy among 18 combinations

The result is notable because the smallest architecture is also the best model by validation accuracy. This shows that increasing depth and parameter count did not automatically improve generalization for this experiment.

8. Confusion Matrix Analysis

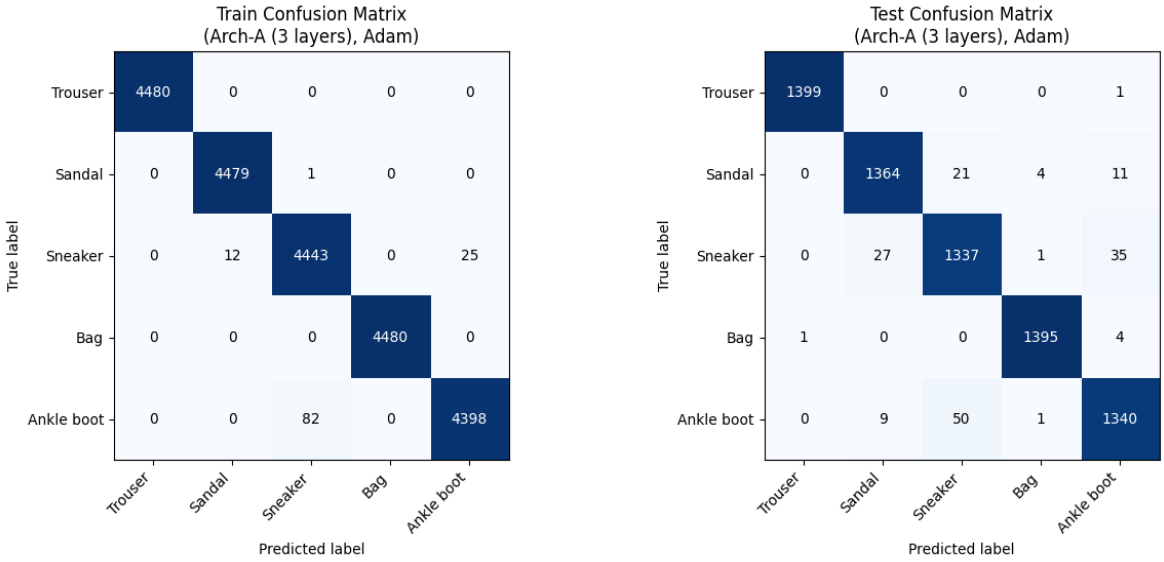


Figure 5. Training and test confusion matrices for the selected Arch-A + Adam model.

8.1 Training confusion matrix

The training matrix is strongly diagonal, showing that the selected model fits the training data very well. Trouser and Bag are classified perfectly in the recorded training matrix. The main training confusion occurs for Ankle boot, where 82 examples are predicted as Sneaker, and for Sneaker, where 12 examples are predicted as Sandal and 25 as Ankle boot.

8.2 Test confusion matrix

True \ Predicted	Trouser	Sandal	Sneaker	Bag	Ankle boot
Trouser	1399	0	0	0	1
Sandal	0	1364	21	4	11
Sneaker	0	27	1337	1	35
Bag	1	0	0	1395	4
Ankle boot	0	9	50	1	1340

The test set contains 7,000 images and the diagonal contains 6835 correct predictions, giving 97.64% overall accuracy. The strongest class-wise performance is for Trouser (99.93% recall) and Bag (99.64% recall). The comparatively lower recalls are for Sneaker (95.50%) and Ankle boot (95.71%).

The most visible confusion is between Sneaker and Ankle boot: 35 Sneaker images are predicted as Ankle boot, while 50 Ankle boot images are predicted as Sneaker. Sandal is also occasionally confused with Sneaker and Ankle boot. These errors are reasonable for visually similar footwear categories and explain why the test accuracy is slightly below the training accuracy.

9. Key Observations

- Adam gives the best overall validation accuracy, 97.36%, when paired with Arch-A.
- The selected Arch-A model is the smallest network, with 110,981 trainable parameters.
- Adding more hidden layers did not improve the best validation accuracy beyond the Arch-A + Adam result.
- Adam converges quickly: 25 epochs for Arch-A and 22 epochs for Arch-B. Momentum is fastest for Arch-C at 27 epochs.
- RMSProp produces very high training accuracy, but its validation accuracy is not consistently the best.
- Batch GD stops after two epochs in the supplied results but remains at 20% accuracy, so epoch count alone is not a sufficient measure of optimizer quality.
- The final test accuracy of 97.64% is slightly higher than the validation accuracy of 97.36%, while remaining consistent with strong generalization.
- The confusion matrix shows that the main remaining classification difficulty is distinguishing visually similar footwear classes, especially Sneaker and Ankle boot.

10. Conclusion

This experiment demonstrates the effect of optimizer choice and FCNN depth on convergence and classification performance for a five-class Fashion-MNIST problem. All non-Batch-GD optimizers achieve strong classification performance, but their convergence speeds and validation accuracies differ. Based on the required validation-based model selection, the best configuration is Arch-A (784 → 128 → 64 → 32 → 5) trained with Adam.

The selected model achieves 99.46% training accuracy, 97.36% validation accuracy and 97.64% test accuracy. Its smaller size also makes it attractive compared with the deeper architectures tested. The experiment therefore shows that a carefully optimized smaller FCNN can outperform deeper alternatives on this selected dataset, while optimizer behavior has a substantial effect on both convergence and final performance.

THANK YOU